

Example 1. While Loop – Sum of Numbers

This example calculates the sum of numbers from 1 to 10 using a `while` loop.

```
public class WhileLoopExample {  
    public static void main(String[] args) {  
        int number = 1, sum = 0;  
  
        while (number <= 10) {  
            sum += number;  
            number++;  
        }  
  
        System.out.println("The sum of numbers from 1 to 10 is: " + sum);  
    }  
}
```

Example 2. Do-While Loop – Input Validation

This example uses a `do-while` loop to ask the user for a positive number and keeps asking until a negative number is entered.

```
import java.util.Scanner;

public class DoWhileExample {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);
        int number;

        do {
            System.out.print("Enter a positive number (negative to stop): ");
            number = scanner.nextInt();
        } while (number >= 0);

        System.out.println("You entered a negative number, exiting...");
        scanner.close();
    }
}
```

Example 3. For Loop – Multiplication Table

This example generates a multiplication table for the number 5 using a `for` loop.

```
public class ForLoopExample {  
    public static void main(String[] args) {  
        int num = 5;  
  
        for (int i = 1; i <= 10; i++) {  
            System.out.println(num + " * " + i + " = " + (num * i));  
        }  
    }  
}
```

Example 4. Break Statement – Exit on Condition

In this example, the loop exits when the counter reaches 5 using the `break` statement.

```
public class BreakExample {  
    public static void main(String[] args) {  
        int sum = 0;  
  
        for (int count = 1; count <= 10; count++) {  
            if (count == 5) {  
                break; // Break the loop when count equals 5  
            }  
            sum += count;  
        }  
  
        System.out.println("The sum is: " + sum);  
    }  
}
```

Example 5. Continue Statement – Skip on Condition

This example demonstrates how the `continue` statement skips the current iteration when the counter is 5.

```
public class ContinueExample {  
    public static void main(String[] args) {  
        int sum = 0;  
  
        for (int count = 1; count <= 10; count++) {  
            if (count == 5) {  
                continue; // Skip the rest of the loop when count equals 5  
            }  
            sum += count;  
        }  
  
        System.out.println("The sum is: " + sum);  
    }  
}
```

Example 6. Label Statement – Break Nested Loops

In this example, the `break` statement is used with a label to break out of both the inner and outer loops.

```
public class LabelExample {  
    public static void main(String[] args) {  
        int counter = 0;  
  
        stop: { // Label the outer loop  
            for (int row = 1; row <= 10; row++) {  
                for (int column = 1; column <= 5; column++) {  
                    if (row == 5) {  
                        break stop; // Break both loops when row equals 5  
                    }  
                    counter++;  
                }  
            }  
        }  
  
        System.out.println("Counter value: " + counter);  
    }  
}
```

Example 7. Count-Controlled Loop (For) – Sum of Even Numbers

This `for` loop calculates the sum of even numbers between 1 and 20.

```
public class EvenSumExample {  
    public static void main(String[] args) {  
        int sum = 0;  
  
        for (int i = 2; i <= 20; i += 2) {  
            sum += i;  
        }  
  
        System.out.println("The sum of even numbers from 1 to 20 is: " + sum);  
    }  
}
```

Example 8. Sentinel-Controlled Loop – Input Numbers Until Sentinel

This `while` loop continues accepting user input until the sentinel value (negative number) is entered.

```
import java.util.Scanner;

public class SentinelLoopExample {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);
        int number;

        System.out.println("Enter positive numbers. Enter a negative number to stop.");

        while ((number = scanner.nextInt()) >= 0) {
            System.out.println("You entered: " + number);
        }

        System.out.println("Loop exited because a negative number was entered.");
        scanner.close();
    }
}
```

Example 9. Nested Loop – Display a Pattern

This example uses nested loops to print a pattern of stars (`\*`).

```
public class NestedLoopExample {  
    public static void main(String[] args) {  
        int rows = 5;  
  
        for (int i = 1; i <= rows; i++) {  
            for (int j = 1; j <= i; j++) {  
                System.out.print("* ");  
            }  
            System.out.println();  
        }  
    }  
}
```

Example 10. Infinite Loop – Until User Breaks

This `while` loop keeps running until the user chooses to exit by entering a negative number.

```
import java.util.Scanner;

public class InfiniteLoopExample {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);
        int number;

        while (true) {
            System.out.print("Enter a number (negative to exit): ");
            number = scanner.nextInt();

            if (number < 0) {
                System.out.println("Exiting the loop.");
                break; // Exits the loop when a negative number is entered
            }
            System.out.println("You entered: " + number);
        }

        scanner.close();
    }
}
```